

102. Binary Tree Level Order Traversal.

Обход двоичного дерева по уровням.

Сложность: Medium · **Паттерн:** обход в ширину с очередью · **Класс:** `block06_trees_graphs.LevelOrderTraversal`.

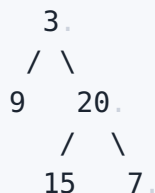
Условие

Дан корень двоичного дерева. Вернуть значения его узлов по уровням: список списков, где первый список — корень, второй — все узлы глубины 2 слева направо, третий — все узлы глубины 3, и так далее.

Примеры.

Пример 1.

Вход: `root = [3, 9, 20, null, null, 15, 7]`.



Выход: `[[3], [9, 20], [15, 7]]`.

Три уровня — три списка. Внутри уровня порядок слева направо: 9 раньше 20, 15 раньше 7.

Пример 2.

Вход: `root = [1]`.

Выход: `[[1]]`.

Пример 3.

```
Вход: root = [].  
Выход: [].
```

Пустое дерево даёт пустой список уровней — не `[[[]]`, а `[]`.

Ограничения.

- число узлов в дереве — от `0` до `2000`.
- `-1000 <= Node.val <= 1000`.

Почему такие ограничения.

Ноль узлов разрешён, поэтому `root == null` — обычный вход с ответом «пустой список». Проверка на `null` в решении есть, и она не охранный: это контракт условия.

Две тысячи узлов — это от одиннадцати уровней у законченного дерева до двух тысяч у цепочки. Откуда эти числа.

Обозначения в формулах ниже. `n` — число узлов в дереве; по условию `n` не больше `2000`. `k` — номер текущего уровня, считая с единицы: у корня `k = 1`, у его потомков `k = 2`, и так далее. `h` — число уровней в дереве, то есть его высота. `nk` — число узлов на уровне `k`. Формула с `k` говорит про один уровень, формула с `h` — про все уровни с первого по `h`-й.

Сколько узлов помещается на уровне. У двоичного дерева каждый узел даёт не больше двух потомков, поэтому уровень не может быть шире удвоенного предыдущего: `n1 = 1` (корень), `n2 ≤ 2`, `n3 ≤ 4`. В общем виде `nk ≤ 2в степени k-минус 1`.

Сколько узлов вмещают первые `h` уровней. Складываем: `n = 1 + 2 + 4 + ... + 2в степени h-минус 1 = 2в степени h - минус 1` — проверить можно на маленьких числах: один уровень вмещает `1 = 21 - минус 1`, два уровня `1 + 2 = 3 = 22 - минус 1`, три уровня `1 + 2 + 4 = 7 = 23 - минус 1`. Каждый новый уровень добавляет столько же узлов, сколько было во всех предыдущих, плюс один, и сумма всегда на единицу меньше следующей степени двойки.

Уровней h .	Вмещают узлов, $n = 2$ в степени h - минус 1.
9.	511.
10.	1023.
11.	2047.
12.	4095.

Минимальное число уровней для 2000 узлов. Ищем по таблице первое h , при котором 2 в степени h - минус 1 ≥ 2000 . Десять уровней вмещают только 1023 узла — мало. Одиннадцать вмещают 2047 — хватает. Значит, ниже одиннадцати уровней дерево из 2000 узлов не уложить. В общем виде $h = \lceil \log_2(n + 1) \rceil$: $\log_2 2001 \approx$ примерно равно 10,97, округление вверх даёт $h = 11$. Таблица и логарифм — один и тот же подсчёт, только логарифм отвечает сразу, без перебора строк.

Сколько узлов на последнем уровне. Первые десять уровней заполнены целиком — 1023 узла. На одиннадцатый остаётся $2000 - 1023 = 977$. Вместить он мог бы $2^{10} = 1024$, так что заполнен он не до конца: $1024 - 977 = 47$ мест пустуют.

Максимальное число уровней. У цепочки каждый узел стоит на своём уровне — $h = n = 2000$. Дерево из 2000 узлов, таким образом, имеет от 11 до 2000 уровней в зависимости от формы; общие формулы собраны в [BinaryTree.md](#), под рис. 1.

Обход в ширину держит в очереди один уровень целиком, поэтому первое дерево — худший случай по памяти, второе — самый лёгкий. Законченное дерево из 2000 узлов проверено тестом.

Диапазон значений на решение не влияет: значения только переписываются в ответ.

Идея решения.

Порядок «уровень за уровнем, внутри уровня слева направо» — это обход в ширину, и очередь даёт его сама: сняли узел с головы, положили его потомков в хвост, и потомки всегда оказываются позади всех узлов своего уровня. Так устроен `levelOrder` в `TreeTraversals`, и он выдаёт плоский список 3, 9, 20, 15, 7.

Но задача просит не плоский список, а список по уровням, и очередь сама по себе не говорит, где кончился один уровень и начался следующий. Ответ: **в начале каждой итерации внешнего цикла очередь содержит ровно один уровень целиком** — и ничего больше. Значит, достаточно запомнить её размер и снять ровно столько узлов: это и есть уровень. Их потомки, которые за это время дописались в хвост, — следующий уровень, и на следующей итерации размер очереди будет равен уже его размеру.

```
public static List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> levels = new ArrayList<>();
    if (root == null) {
        return levels;
    }

    Queue<TreeNode> nodesToVisit = new ArrayDeque<>();
    nodesToVisit.add(root);
    while (!nodesToVisit.isEmpty()) {
        int nodesOnLevel = nodesToVisit.size();
        List<Integer> levelValues = new ArrayList<>(nodesOnLevel);
        for (int taken = 0; taken < nodesOnLevel; taken++) {
            TreeNode node = nodesToVisit.remove();
            levelValues.add(node.val);
            if (node.left != null) {
                nodesToVisit.add(node.left);
            }
            if (node.right != null) {
                nodesToVisit.add(node.right);
            }
        }
        levels.add(levelValues);
    }
    return levels;
}
```

Что происходит в коде, строка за строкой.

Оба цикла работают с одной и той же очередью `nodesToVisit`; список `levels` — только приёмник, по нему никто не идёт, в него лишь дописывают. «Уровень» в коде нигде не хранится как отдельная сущность: уровнем оказывается то, что лежит в `nodesToVisit` в момент, когда внешний цикл начинает очередную итерацию. Это и надо увидеть.

`nodesToVisit.add(root)` — перед первой итерацией внешнего цикла в очереди один узел, корень. Корень — это весь первый уровень.

`while (!nodesToVisit.isEmpty())` — внешний цикл повторяется, пока в очереди что-то есть. Он не знает про уровни; он знает только, что очередь не пуста.

`int nodesOnLevel = nodesToVisit.size()` — первая строка итерации внешнего цикла: сколько узлов лежит в очереди прямо сейчас. На первой итерации это `1`. Дальше будет показано, что в этот момент в очереди всегда лежит ровно один уровень, поэтому число и названо `nodesOnLevel`.

`List<Integer> levelValues = new ArrayList<>(nodesOnLevel)` — пустой список под значения этого уровня.

`for (int taken = 0; taken < nodesOnLevel; taken++)` — внутренний цикл: снять из очереди ровно `nodesOnLevel` узлов. Не «пока очередь не опустеет», а ровно столько, сколько насчитали в начале итерации внешнего цикла.

`TreeNode node = nodesToVisit.remove()` — сняли узел с головы. `levelValues.add(node.val)` — его значение записали в список текущего уровня. Затем два `if`: если у узла есть левый потомок — в хвост очереди; если правый — тоже в хвост. Потомки встали в очередь позади всех, кто там уже был.

`levels.add(levelValues)` — внутренний цикл закончился, список уровня готов, его кладём в ответ. Это единственное место, где `levels` меняется.

Теперь почему на следующей итерации внешнего цикла в `nodesToVisit` снова лежит ровно один уровень. Внутренний цикл снял `nodesOnLevel` узлов — все, что были в очереди в начале итерации внешнего цикла, то есть весь текущий уровень. Добавил он только их потомков. Значит, после внутреннего цикла в очереди нет ни одного узла текущего уровня и есть все узлы следующего — каждый узел следующего уровня чей-то потомок, а всех родителей мы только что сняли. Очередь снова содержит ровно один уровень, и `nodesToVisit.size()` на новой итерации даст его размер.

Так «внешний цикл идёт по уровням» получается не потому, что где-то лежит список уровней, а потому, что каждая итерация внешнего цикла снимает из очереди один уровень целиком и оставляет в ней следующий.

Что делает каждая переменная.

Переменная	Что хранит	Зачем она нужна
<code>root</code>	корень дерева, может быть <code>null</code> .	вход задачи; имя из условия LeetCode.
<code>levels</code>	ответ: список списков, по одному на уровень.	заполняется по одному уровню за итерацию внешнего цикла.
<code>nodesToVisit</code>	очередь узлов, чьих потомков ещё не смотрели.	сердце обхода в ширину: FIFO гарантирует, что уровни идут по порядку, а внутри уровня — слева направо.
<code>nodesOnLevel</code>	сколько узлов лежит в очереди в начале итерации внешнего цикла.	размер текущего уровня, снятый ДО того, как в очередь пойдут потомки; без него уровни слились бы в один плоский список.
<code>levelValues</code>	значения узлов текущего уровня.	собирается внутренним циклом и целиком кладётся в <code>levels</code> .
<code>taken</code>	сколько узлов уже снято на этой итерации внешнего цикла.	счётчик внутреннего цикла: остановиться после <code>nodesOnLevel</code> узлов, а не когда очередь опустеет.
<code>node</code>	узел, снятый с головы очереди.	его значение идёт в <code>levelValues</code> , его потомки — в хвост очереди.

Имя `nodesToVisit` — «узлы, которые предстоит посетить»: они уже обнаружены как потомки, но ещё не сняты с очереди и не записаны в ответ. В `TreeNode.fromLevelOrder` очередь названа так же, но «посетить» там значит другое: снятому узлу выдают двух потомков из записи.

Почему размер очереди снимается до внутреннего цикла.

Внутренний цикл одновременно и снимает узлы с головы, и кладёт их потомков в хвост, поэтому `nodesToVisit.size()` меняется на каждом шаге. Если бы условие было `taken < nodesToVisit.size()`, оно пересчитывало бы размер на каждой итерации, и цикл продолжался бы, пока очередь не опустеет, — то есть прошёл бы всё дерево одним уровнем. Снятое заранее число `nodesOnLevel` не меняется, и цикл останавливается ровно на границе уровня.

Это приём 10 из шпаргалки: цикл сам меняет то, по чему идёт, поэтому граница снимается до входа в него.

Почему узлы одного уровня у разных родителей не перепутаются.

Пусть уровень — это 2 и 3, у 2 есть потомок 4, у 3 — потомок 5. Внутренний цикл снимает 2, кладёт 4; снимает 3, кладёт 5. В очереди 4, 5 — оба потомка разных родителей, в порядке слева направо, потому что родители снимались слева направо. На следующей итерации внешнего цикла `nodesOnLevel = 2`, и они попадают в один список. Разделение на уровни держится не на родителях, а на моменте, когда снят размер очереди.

Тест «узлы одного уровня у разных родителей» проверяет ровно это; ошибка «уровень = потомки одного родителя» на нём падает.

Разбор по шагам.

Пример 1, дерево `[3, 9, 20, null, null, 15, 7]`. Очередь показана после каждой итерации внешнего цикла:

старт	очередь: 3	levels: [].
итерация 1, размер 1 сняли 3, положили 9, 20		levels: [[3]].
	очередь: 9, 20.	
итерация 2, размер 2 сняли 9 (потомков нет),		
	сняли 20, положили 15, 7	levels: [[3], [9, 20]].
	очередь: 15, 7.	
итерация 3, размер 2 сняли 15, сняли 7,		
	потомков нет	levels: [[3], [9, 20], [15,
		7]].
	очередь пуста — выход.	

На итерации 2 после снятия 9 очередь выглядит как 20, после снятия 20 — как 15, 7. Размер очереди за итерацию менялся четыре раза (2 → затем 1 → затем 0 → затем 1 → затем 2), но `nodesOnLevel` остался равен двум, и 15 с 7 в уровень [9, 20] не попали.

Свойства решения.

Что означает каждая строка — в `docs/algorithm-properties.md`.

Характеристика.	Значение.
Время	<code>0(n)</code> : каждый узел один раз кладётся в очередь и один раз снимается; лучший и худший случай совпадают.
Память.	<code>0(w)</code> сверх ответа, где <code>w</code> — наибольшее число узлов на одном уровне: столько узлов одновременно лежит в очереди. У законченного дерева это около <code>n / 2</code> , у цепочки — 1. Сам ответ — <code>0(n)</code> , он обязателен.
Проходов по входу.	один.
Ранний выход.	нет: ответ включает все узлы.
Изменяет ли вход.	нет, дерево только читается.
Требования к входу.	никаких: пустое дерево — законный вход.
Работа на потоке.	нет: нужен доступ к потомкам по ссылкам.
Детерминированность.	полная.
Завершаемость.	каждый узел попадает в очередь ровно один раз, потому что у него один родитель; очередь опустеет после <code>n</code> снятий.
Поведение на вырожденных входах.	пустое дерево — <code>[]</code> , один узел — <code>[[1]]</code> ; законченное дерево из 2000 узлов даёт 11 уровней.

Приёмы из шпаргалки.

- **Приём 10** (цикл меняет то, по чему идёт, — зафиксируй границу до входа) — `nodesOnLevel` снимается, пока размер очереди означает «один уровень»; читать `nodesToVisit.size()` внутри цикла нельзя, он уже включает потомков.
- **Приём 9** (каждая итерация обязана уменьшать задачу) — внешний цикл на каждой итерации снимает целый уровень, а узлов конечное число; внутренний ограничен счётчиком, а не состоянием очереди.

Опора в книге.

Глава 4 «Деревья и графы» (Часть IX «Вопросы собеседования», с. 92 / Часть X «Решения», с. 248), задача 4.3 «Список глубин»: для каждого уровня дерева построить связный список его узлов. Это та же задача, только результат — списки узлов, а не значений. В книге даны два решения: обход в глубину с передачей номера уровня и обход в ширину, где уровни разделены иначе — два списка, текущий уровень и следующий, вместо одной очереди со снятым размером. Идея та же: на уровень `i` переходят, только когда весь уровень `i - минус 1` разобран. В RAG-файле названия задач потеряны при распознавании; искать по фразе «создает связный список всех узлов, находящихся на каждой глубине».

См. также.

- Визуализация: `docs/visualizations/block06_trees_graphs/LevelOrderTraversal.html`.
- `docs/datastructures/BinaryTree.md` — обход в ширину, память `O(w)` против `O(h)`.
- `docs/datastructures/Queue.md` — очередь, `ArrayDeque`, роль FIFO в обходе в ширину.
- `TreeNotation.md` — `fromLevelOrder` и `toLevelOrder`: тот же обход, но в плоскую запись.
- `MaxDepthBinaryTree.md` — обход в глубину для сравнения; число уровней в ответе этой задачи равно `maxDepth`.

- Характеристики решения: `docs/algorithm-properties.md`.
- LeetCode: <https://leetcode.com/problems/binary-tree-level-order-traversal/>.